Manual técnico de SomnGuard

Operación, configuración y mantenimiento
Edición 1.0 · 9 de octubre de 2026
Dirigido a: personal de desarrollo, DevOps, base de datos y operación del dispositivo

Contenido

1 Sobre este manual

2 SomnGuard de un vistazo

3 Requisitos previos

4 Preparar y arrancar el entorno local

5 Configuración y secretos

6 Operación y comprobaciones

7 API y seguridad

8 Dispositivo de borde

9 Mantenimiento del código y la base

10 Respaldo, recuperación y actualización

11 Diagnóstico de problemas

12 Referencia rápida y documentación relacionada

1. Sobre este manual

Este manual reúne instrucciones técnicas para trabajar con los repositorios actuales de SomnGuard. Cubre configuración y desarrollo local, comprobaciones operativas, API, migraciones, streaming y dispositivo edge. No certifica un despliegue productivo: la documentación del proyecto aún deja abiertas la topología productiva, la recuperación, la observabilidad y la distribución final.

La revisión se hizo por inspección estática el 9 de octubre de 2026. Los comandos de este documento provienen de archivos de configuración o documentación de cada repositorio; no se ejecutaron contra el sistema. Antes de usarlos en QA, valide los valores de ambiente y el directorio desde el que corre cada comando.

1.1 Audiencia y conocimientos previos

Se supone familiaridad con una terminal Bash o Git Bash, Git, Docker Compose, HTTP y variables de entorno. Las tareas de API requieren Java y Maven; web/app, Node y npm; y dispositivo, Python y acceso al hardware. Las operaciones contra QA o producción requieren autorización y procedimientos específicos del equipo responsable.

1.2 Convenciones y cuidado de datos

- Los bloques de comandos indican el repositorio desde el que deben ejecutarse. Los ejemplos usan Bash/Git Bash.
- Las rutas, variables, perfiles y endpoints se escriben con su nombre real en el código.
- Use `.env.example` como plantilla y mantenga los valores reales fuera de Git. Los valores de ejemplo no son secretos seguros para ambientes compartidos.
- No publique JWT, claves privadas, API keys, tokens de provisión, datos personales ni evidencia de conductores.
- No use `docker compose down -v`, rollbacks, borrados de identidad del dispositivo ni procedimientos de reset contra datos compartidos sin plan de recuperación aprobado.

2. SomnGuard de un vistazo

SomnGuard combina procesamiento local en el dispositivo, una API central, persistencia relacional, almacenamiento de evidencia y dos clientes. Los directorios son repositorios hermanos; no hay un Compose en la raíz que levante toda la solución.

Unidad	Repositorio	Responsabilidad	Inicio habitual
Base de datos y migraciones	`somnguard-db/`	PostgreSQL 16, seis esquemas y cambios Liquibase	Compose propio; Liquibase se ejecuta como tarea aparte
Backend	`somnguard-api/`	API Spring Boot, JWT, permisos, dispositivos, eventos, notificaciones y streaming	Maven o Compose propio

Unidad	Repositorio	Responsabilidad	Inicio habitual
Evidencia local de desarrollo	`somnguard-api/`	S3Mock compatible con S3; persiste en volumen Compose	Se levanta con el Compose del API
Video opcional	`somnguard-api/` y dispositivo	SFU LiveKit y, opcionalmente, Coturn/TURN	Perfil Compose `livekit` más configuración y paquetes de streaming
Portal	`somnguard-portal/`	Cliente web React/Vite	Servidor Vite local
App móvil	`somnguard-app/SomnGuard-app/`	Cliente Expo/React Native; video nativo LiveKit/relay	Expo para desarrollo; EAS para APK con módulos nativos
Dispositivo edge	`somnguard-device/`	Cámara, visión, alertas locales, heartbeat, buffer y sincronización	Python en Windows, Linux o Raspberry Pi OS según el README

2.1 Flujo de datos

1. El dispositivo procesa imagen localmente, registra eventos y los envía a la API. Durante una pérdida de red, la lógica edge conserva eventos pendientes en SQLite y los sincroniza al recuperar conexión.
2. La API autentica usuarios/dispositivos, aplica reglas de negocio y persiste metadatos en PostgreSQL. Las imágenes y otros medios se guardan en almacenamiento S3 compatible.
3. Portal y app consumen la API. Para video, la ruta principal usa LiveKit; existe un relay WebSocket como alternativa. El servicio de video se configura por separado.

2.2 Direcciones y puertos locales

Servicio	Dirección por omisión o ruta	Condición
API	`http://localhost:8080`	Puerto por omisión de Spring/ejemplo Compose
Health API	`/actuator/health`	Actuator; no `/health` según la configuración actual
OpenAPI UI	`/swagger-ui.html`	Habilitada por Springdoc en la configuración revisada
PostgreSQL	`localhost:5432`	Compose de DB; cambiar mediante `POSTGRES_PORT`
S3Mock	`localhost:9000`	Desarrollo; el contenedor escucha internamente en `9090`
Portal Vite	`localhost:5173`	Valor por omisión de Vite si el puerto está libre
Expo	`localhost:3000`	El script del proyecto solicita ese puerto
LiveKit	TCP `7880`, TCP `7881`, UDP `7882`	Solo con perfil `livekit`
Relay	WebSocket `/ws/stream`	Servido por API; requiere un dispositivo publicando video
Coturn	`3478` y rango UDP `49160-49200`	Solo cuando se necesita atravesar NAT; revisar Compose y firewall

No exponga puertos de desarrollo directamente a Internet. La dirección que anuncia LiveKit debe ser accesible desde el teléfono y el dispositivo; `localhost` solo vale para procesos del mismo host.

3. Requisitos previos

Trabajo	Requisitos declarados en el repositorio
Contenedores y DB	Docker con Docker Compose; PostgreSQL 16 en contenedor
Backend	Java 21 y Maven; Maven Wrapper no figura en la raíz del API
Portal	Node/npm; versiones de referencia en `package.json` y lockfile
App	Node/npm, Expo SDK 57 y EAS CLI para builds internos; LiveKit requiere cliente nativo
Dispositivo	Python 3.11+, cámara USB o CSI, parlante para alertas; dependencias de visión y modelos del proyecto
Video edge	Extra Python `[stream]` y configuración de API/LiveKit

Docker, Compose y Node se consultaron en el host de esta revisión (Docker 29.2.1, Compose 5.0.2, Node 24.14.1). Esas versiones describen el host de documentación; no son mínimos garantizados para el proyecto. La versión efectiva de Java, Maven, PostgreSQL en QA/producción, Android/iOS y hardware homologado queda por validar en sus ambientes.

4. Preparar y arrancar el entorno local

Los siguientes pasos corresponden a desarrollo y mantienen las unidades separadas. No constituyen un procedimiento de release.

4.1 PostgreSQL y Liquibase

Desde `somnguard-db/`, prepare el archivo indicado por el setup local con valores de desarrollo propios. No use las contraseñas de ejemplo para datos compartidos.

```
cp .env.example .env.develop
# Edite .env.develop y configure usuario, contraseña, base y puerto de desarrollo.
docker compose --env-file .env.develop up -d postgres
docker compose --env-file .env.develop run --rm liquibase update
```

El contenedor de Liquibase utiliza `changelog/changelog-master.yaml`. El estado de migraciones se consulta con:

```
docker compose --env-file .env.develop run --rm liquibase status --verbose
```

4.2 API y almacenamiento de evidencia

El Compose de `somnguard-api/` inicia el API y S3Mock. PostgreSQL debe estar disponible por separado. Copie `.env.example` a `.env`, haga coincidir URL/puerto/nombre/usuario/contraseña con la DB anterior y revise las rutas de claves antes de arrancar.

```
cd ../somnguard-api
cp .env.example .env
# Edite .env con valores locales; nunca reutilice secretos de ejemplo en producción.
docker compose --env-file .env up -d --build
```

Compruebe `http://localhost:8080/actuator/health` y abra `http://localhost:8080/swagger-ui.html`. El README del API indica `/health`, pero la configuración activa y `SecurityConfig` usan `/actuator/health`.

Para video con LiveKit, defina la URL, dirección anunciada y claves locales compartidas entre API y LiveKit; habilite el perfil:

```
docker compose --env-file .env --profile livekit up -d --build
docker compose --env-file .env --profile livekit logs -f livekit
```

LiveKit no se incluye en el arranque normal. Coturn usa el mismo perfil; su utilización, los puertos públicos y TLS/WSS requieren configuración de red aprobada. S3Mock es almacenamiento de desarrollo, no una definición de almacenamiento productivo.

4.3 Portal web

Desde `somnguard-portal/`, configure `VITE_API_URL` con la base del API y el prefijo `/api/v1`. El ejemplo del repositorio es local.

```
npm ci
npm run dev
```

Para preparar el paquete web, los scripts disponibles son `npm run lint` y `npm run build`. No se encontró un script `test` en `package.json`.

4.4 App móvil

Desde `somnguard-app/SomnGuard-app/`, configure `EXPO_PUBLIC_API_URL` como origen del API, sin `/api/v1` y sin barra final. Para un teléfono físico use una IP o dominio accesible desde su red; el `localhost` del teléfono apunta al teléfono.

```
npm ci
npm run start
```

Expo Go no lleva el módulo nativo LiveKit. Para probar streaming se requiere el cliente de desarrollo o el APK interno configurado por EAS:

```
npm run build:android:apk
```

El comando utiliza el perfil EAS `preview` y necesita EAS, credenciales y valores del ambiente adecuados. El perfil `production` no define `EXPO_PUBLIC_API_URL` en el `eas.json` revisado; no asuma que está listo para publicación.

4.5 Dispositivo de borde en modo de desarrollo

El README del dispositivo describe ejecución directa. No use el proceso local como instalación de campo ni almacene credenciales en un repositorio.

```
cd ../somnguard-device
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[stream]"
python scripts/download_models.py
```

Configure `.env` a partir de `.env.example`, comenzando con `SOMNGUARD_API_URL` accesible desde el dispositivo. Ejecute `python -m app.main` para iniciar el proceso local. En Windows, siga los comandos PowerShell de `somnguard-device/README.md`.

El primer aprovisionamiento requiere un token temporal emitido por el flujo administrativo. Tras el registro inicial, el dispositivo guarda su ID y API key en `data/device_identity.json`. Proteja ese directorio como secreto de dispositivo. Los scripts `provision.sh`, `install_service.sh`, `update.sh` y `factory_reset.sh` están vacíos en el commit revisado: no se han de usar como instaladores automatizados.

5. Configuración y secretos

Cada repositorio consume su propio archivo de entorno. No cree un `.env` común ni copie un secreto entre ambientes.

Componente	Variables o archivos a revisar	Precaución
DB	`POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_PORT`, `COMPOSE_PROJECT_NAME`	Asegure que la URL del API coincide con esta instancia. Mantenga el volumen de datos.
API	`SPRING_PROFILES_ACTIVE`, `SERVER_PORT`, `SPRING_DATASOURCE_URL`, usuario/contraseña DB, rutas de JWT, CORS	Claves RSA de `keys/dev` son solo desarrollo; prod debe montar secretos protegidos.
Correo y avisos	`BREVO_*`, `NOTIFICATION_PUSH_PROVIDER`, `FCM_SERVICE_ACCOUNT_*`	Push usa `log` por omisión; eso no equivale a entrega push móvil.
Evidencia	`S3_ENDPOINT`, `S3_REGION`, `S3_BUCKET`, credenciales S3	Compose local usa S3Mock; en ejecución desde host la URL difiere de la URL entre contenedores.
Video	`LIVEKIT_ENABLED`, `LIVEKIT_URL`, claves LiveKit, TTL, `LIVEKIT_NODE_IP`, datos Coturn	Use dirección alcanzable y secretos propios; no anuncie loopback a clientes remotos.
Portal	`VITE_API_URL`	Incluye `/api/v1`. El valor queda compilado en el cliente web; no poner secretos aquí.
App	`EXPO_PUBLIC_API_URL`	Es pública y apunta al origen API sin prefijo. Defina valor distinto por build.
Edge	`SOMNGUARD_API_URL`, token temporal de provisión, ID/API key, `SOMNGUARD_DATA_DIR`, nivel de log y streaming	Nunca registre el token; proteja la identidad local y el archivo `.env`.

La plantilla de API tiene rutas de claves para perfiles de desarrollo y producción. No monte una clave privada de desarrollo en QA o producción. En dispositivos compartidos y servicios administrados, use el mecanismo de secretos aprobado por la organización.

6. Operación y comprobaciones

6.1 Estado y registros

Ejecute estos comandos en el repositorio Compose correspondiente:

```
# En somnguard-db
docker compose --env-file .env.develop ps

# En somnguard-api
docker compose --env-file .env logs -f api
docker compose --env-file .env logs -f minio
```

La comprobación HTTP del API es `GET /actuator/health`. En QA/producción los detalles de salud dependen del perfil y pueden quedar limitados a usuarios autorizados. `docker compose ps` muestra el estado del contenedor de PostgreSQL; el Compose de API no contiene una instancia PostgreSQL.

6.2 Salud del edge

El proceso edge envía heartbeat y detecta conectividad periódicamente. Si pierde contacto con el servidor, el estado remoto puede aparecer desconectado, mientras el procesamiento local continúa y los eventos quedan pendientes de sincronización. El almacenamiento local y la evidencia también deben vigilarse; el directorio `data/` puede incluir credenciales y medios sensibles.

6.3 Parada

Use `docker compose stop` desde el repositorio de la unidad que desea detener. `docker compose down` retira los contenedores de ese proyecto, según el Compose usado. No añada `-v`: puede borrar la base PostgreSQL o el volumen de evidencia. Para desarrollo local, `Ctrl+C` termina el proceso de portal, Expo o edge en primer plano.

7. API y seguridad

La API usa prefijo `/api/v1`. Swagger UI en `/swagger-ui.html` y el JSON OpenAPI bajo `/v3/api-docs` describen métodos, modelos, campos y respuestas vigentes. Consulte ese contrato en el build que corresponda al ambiente antes de integrar clientes.

Grupo	Ruta base	Control habitual
Autenticación/cuenta	`/api/v1/auth`, `/api/v1/account`	Login/registro/verificación/refresh; operaciones de cuenta autenticadas según método
Dispositivos y configuración	`/api/v1/devices`	JWT con permisos para administración; identidad de dispositivo para heartbeat/telemetría
Streaming	`/api/v1/devices/{id}/stream`	Sesión autenticada; URL/credenciales de video con vigencia limitada
Telemetría y eventos	`/api/v1/telemetry`, `/api/v1/events`	API key del dispositivo para ingestión y JWT/permisos para consulta
Catálogos	`/api/v1/catalogs`	JWT y permisos del módulo
Notificaciones/RBAC	`/api/v1/notifications`, rutas de seguridad bajo `/api/v1`	JWT y rol/permisos administrativos según la operación
Salud y documentación	`/actuator/health`, `/swagger-ui.html`, `/v3/api-docs`	El filtro de seguridad permite health y documentación; detalles Actuator dependen del perfil

Las sesiones de usuario utilizan JWT RS256. El dispositivo presenta `X-Device-ID` y `X-API-Key`; el token de provisión se reserva para el registro inicial. No use credenciales de desarrollador en Swagger o instalaciones compartidas. Las formas exactas de login, refresco, registro, heartbeat, eventos y streaming se consultan en OpenAPI, no se copian a mano aquí.

8. Dispositivo de borde

8.1 Estructura y ciclo local

El código separa configuración/identidad, manejo del dispositivo, captura, análisis, almacenamiento SQLite, sincronización y publicación de video bajo `somnguard-device/app/`. `config/device.default.json` aporta valores base; el dispositivo puede guardar caché y cambios locales en `data/`.

El buffer conserva eventos pendientes y el motor de sincronización envía lotes cuando vuelve la conectividad. El código informa que un modo `OFFLINE` continúa detectando. Un comentario antiguo del encabezado de `app/device/manager.py` dice lo contrario; la implementación en `_enter_offline` y el flujo actual describen continuidad local. Si aparecen discrepancias, verifique el comportamiento en hardware de QA y corrija la documentación fuente.

8.2 Video y modelos

La publicación LiveKit requiere la extra Python `[stream]`, el servicio habilitado y URL/puertos alcanzables. El relay usa WebSocket servido por API. Sin LiveKit, el lector puede depender del relay; un estado de sesión creada no garantiza que haya frames disponibles.

El README indica que la detección de cinturón está desactivada por configuración/modelo hasta la validación de hardware. Los modelos y los resultados deben comprobarse para el dispositivo homologado; no infiera que todos los detectores descritos en requisitos estén activos en producción.

9. Mantenimiento del código y la base

9.1 Comandos disponibles por repositorio

Estos comandos están definidos por scripts y workflows. No se ejecutaron en esta revisión.

```
# API: mismos pasos base que CI
mvn -B clean compile
SPRING_PROFILES_ACTIVE=test mvn -B test
mvn -B checkstyle:check

# Portal
npm ci
npm run lint
npm run build

# App móvil
npm ci
npm run lint

# Dispositivo (Linux/macOS/Git Bash con Make y Python preparado)
make lint
make test
```

La API también requiere claves JWT de prueba en el workflow CI. El dispositivo puede probarse en Windows siguiendo su README y `py -m pytest tests/ -v`. La app no declara un script de pruebas en su `package.json`; no sustituya esta ausencia por un comando inventado.

9.2 Arquitectura y cambios

El API es un monolito modular con puertos y adaptadores. Los paquetes principales incluyen `security`, `parameterization`, `device_management`, `telemetry_service`, `monitoring` y `analytics`. Las entidades de persistencia y los controladores se agrupan por módulo. Mantenga reglas del dominio fuera de la capa web y use los puertos/adaptadores existentes al ampliar casos de uso.

Portal y app separan rutas/composición, funcionalidades y componentes compartidos. En móvil, las rutas están en `SomnGuard-app/src/app/`; la lógica se organiza bajo `src/features/`. Edge organiza los ciclos de cámara, análisis, almacenamiento y sincronización bajo `app/`.

9.3 Cambios de esquema

La fuente versionada del esquema es `somnguard-db/changelog/changelog-master.yaml`. Los changesets se separan por DDL, DML, DCL y TCL y se agrupan en módulos. Use `liquibase status --verbose` para inspeccionar pendientes. La documentación del proyecto permite rollback en desarrollo y declara QA/main forward-only; ante un cambio defectuoso en esos ambientes corresponde un forward-fix aprobado o recuperar desde respaldo.

No aplique SQL manual como sustituto del changelog. Resuelva antes la diferencia entre Liquibase y `ddl-auto:update` del perfil API `dev`; no apunte ese perfil a una base compartida.

10. Respaldo, recuperación y actualización

El estado persistente incluye PostgreSQL y los objetos de evidencia S3; respaldar uno solo puede dejar referencias rotas. El volumen local de S3Mock se llama `s3mock-data`; PostgreSQL usa `postgres_data`. Las estrategias documentales dejan sin confirmar cifrado/destino, frecuencia, PITR, RTO/RPO y restore drill.

La guía `docs/13-operations/backup-and-recovery.md` ofrece propuestas de dump/restauración, pero no hay evidencia aquí de que se hayan probado contra este despliegue ni de que la ruta de salida esté montada. Antes de usarla, defina un destino persistente y protegido, respalde DB y medios de forma coordinada y ensaye la restauración en un entorno efímero.

No restaure sobre producción como diagnóstico.

El repositorio API publica una imagen mediante GitHub Actions desde ramas configuradas, pero eso no equivale a un release coordinado de API, DB, portal, app y edge. Revise compatibilidad de migraciones, variables y clientes por ambiente y siga el proceso de release aprobado. No existe un único comando de actualización para todo el sistema.

11. Diagnóstico de problemas

Síntoma	Comprobaciones iniciales
API no conecta a PostgreSQL	Confirme que el Compose de DB está activo, puerto y credenciales coinciden y el host corresponde al proceso: `localhost` desde el host, nombre de servicio o `host.docker.internal` desde contenedor según red/plataforma.
API no inicia por datasource o esquema	Consulte los logs del API; compare URL/usuario/perfil y cambios Liquibase aplicados. No resuelva ejecutando un rollback en QA/producción.
`/health` devuelve 404	Use `/actuator/health`, que corresponde al `management.endpoints.web.base-path` actual.
Portal devuelve error API	Revise `VITE_API_URL`, CORS y disponibilidad de API; el portal espera base URL con `/api/v1`.
Móvil no alcanza API en teléfono físico	Cambie `localhost` por una IP/DNS accesible desde el teléfono y configure CORS/red donde aplique. La app espera el origen sin `/api/v1`.
No aparece video	Compruebe si LiveKit está iniciado con perfil `livekit`, `LIVEKIT_ENABLED`, URL anunciada, puertos/firewall, extra `[stream]` del edge y publicación de frames. En Expo Go no está el módulo nativo.
Edge aparece offline	Revise URL del API, heartbeat, reloj/red y espacio en disco. La detección local puede continuar y sincronizar más tarde; cuide el buffer y los medios locales.
Evidencia falla aunque exista evento	Verifique conectividad y servicio/credenciales S3; metadatos y bytes se almacenan en componentes distintos.
No funciona `make provision`, `make update` o `make factory-reset`	Los scripts `.sh` asociados están vacíos en el baseline revisado; use solo el procedimiento manual aprobado y no suponga que el objetivo Makefile realiza la operación.
Healthcheck o manual antiguo discrepa	Priorice rutas/configuración del código ejecutado, contraste perfil/commit y actualice el documento fuente que quedó atrás.

12. Referencia rápida y documentación relacionada

Para	Comando o referencia
Levantar PostgreSQL local	`cd somnguard-db && docker compose --env-file .env.develop up -d postgres`
Aplicar migraciones	`cd somnguard-db && docker compose --env-file .env.develop run --rm liquibase update`
Ver salud de API	`http://localhost:8080/actuator/health`
Ver contrato API	`http://localhost:8080/swagger-ui.html`
Levantar API y S3Mock	`cd somnguard-api && docker compose --env-file .env up -d --build`
Levantar streaming opcional	`cd somnguard-api && docker compose --env-file .env --profile livekit up -d --build`
Iniciar portal	`cd somnguard-portal && npm run dev`
Iniciar Expo	`cd somnguard-app/SomnGuard-app && npm run start`
Ejecutar edge	`cd somnguard-device && python -m app.main`

Los comandos de inicio se ofrecen para entornos locales y presuponen `.env` configurados. No los copie como runbook de producción.

Documentos relacionados

- `somnguard-docs/docs/05-architecture/architecture-document.md`: vista arquitectónica y decisiones.
- `somnguard-docs/docs/07-api-design/`: convenciones y autenticación; contraste contratos con Swagger actual.

- `somnguard-docs/docs/06-data-architecture/migration-strategy.md`: estructura de changelogs.
- `somnguard-docs/docs/10-devops/local-setup.md` y `environments.md`: flujo de desarrollo/ambientes; algunas instrucciones requieren actualización.
- `somnguard-docs/docs/13-operations/backup-and-recovery.md`, `observability.md` e `incident-management.md`: estrategia aún con decisiones pendientes.
- `somnguard-device/README.md`, `somnguard-api/README.md`, `somnguard-portal/README.md` y `somnguard-app/README.md`: requisitos y comandos de cada repositorio; contrastar con el commit actual.
- Informe, matriz, índice y pendientes.

Pendientes de decisión para un runbook productivo: despliegue y TLS integrados; secretos y CORS por ambiente; resolución de Liquibase frente a Hibernate en desarrollo; backups probados; monitoreo/guardia; perfiles de build móvil; configuración pública de LiveKit/TURN; scripts edge operativos; y soporte de hardware/modelos.